Document Fingerprinting — Microsoft Purview

Implementation Solution Guide

Version 1.0.0 · 2026-07-07

Contents

Executive Summary — Document Fingerprinting (Microsoft Purview) 4

The problem . . . 4

The solution . . . 4

What it delivers . . . 4

Business outcomes . . . 4

Boundaries (so scope is clear) . . . 5

1. Solution Overview — Document Fingerprinting (Microsoft Purview) 5

What this solution is . . . 5

Why this matters . . . 6

How Document Fingerprinting works . . . 6

What a fingerprint can and cannot detect (be honest with buyers) . . . 7

Naming convention (used in every doc and script) . . . 7

The standard deployment model . . . 7

2. Requirements — Document Fingerprinting (Microsoft Purview) 8

2.1 Licensing sufficient for DLP and document fingerprinting . . . 8

2.2 Fingerprint/DLP-specific least-privilege roles . . . 8

2.3 The source form templates to fingerprint (blank/representative, text-based) . . . 9

2.4 A form inventory and classification intent . . . 9

2.5 A form-version refresh process . . . 9

2.6 A pilot ring, simulation dwell time, and change control (DLP-specific) . . . 10

2.7 An incident/alert triage owner and destination . . . 10

Global Requirements — Microsoft Purview Solutions 11

1. Microsoft 365 tenant with the required Purview/compliance licensing . . . 11

2. Purview roles (least-privilege) and portal access . . . 12

3. Security & Compliance PowerShell + Graph toolchain . . . 12

4. Data sources & scope in scope . . . 12

5. Compliance, records, privacy & legal stakeholders . . . 13

6. A pilot scope and change control . . . 13

3. Implementation Guide — Document Fingerprinting (Microsoft Purview) 13

Phase 0 — Connect and baseline (Assess) . . . 14

Phase 1 — Assemble and validate the source forms (Design) . . . 14

Phase 2 — Create the fingerprints and the SIT (Configure) . . . 14

Phase 3 — Build the DLP policy in SIMULATION (Configure — no user impact) . . . 15

Phase 4 — Measure and confirm detection (Pilot — simulate) . . . 16

Phase 5 — Enforce in a pilot ring, then expand (Pilot → Enforce) . . . 16

Order-of-operations summary . . . 17
5. Testing & Validation — Document Fingerprinting (Microsoft Purview) 18
Test A — Detection (the forms are detected) . . . 18
Test B — Specificity / false-positive rate (adoption gate) . . . 19
Test C — Incident / alert handling (the control is operable) . . . 19
Simulation-first & rollback validation . . . 20
Evidence to capture (for the deliverable record) . . . 21
6. Troubleshooting — Document Fingerprinting (Microsoft Purview) 21
Creating the fingerprint / SIT . . . 21
Detection (form not matched) . . . 22
Over-matching (false positives) . . . 22
Alerts / incident handling . . . 22
Refresh / drift . . . 23
Connectivity / tooling . . . 23
7. Best Practices — Document Fingerprinting (Microsoft Purview) 24
Fingerprint the right source . . . 24
Simulate before you enforce — always . . . 24
Keep detection specific (protect adoption) . . . 25
Refresh fingerprints as forms change . . . 25
Right-size the user experience . . . 25
Classification hygiene . . . 25
Operations & governance . . . 26
Stay inside the boundary . . . 26
8. Deliverables Summary — Document Fingerprinting (Microsoft Purview) 26
Documentation . . . 26
PowerShell tools (5) — Security & Compliance PowerShell . . . 27
Definition of done . . . 28
Appendix A — PowerShell Scripts 29

Executive Summary — Document Fingerprinting (Microsoft Purview)

The problem

Most sensitive business runs on **standard forms and templates** — onboarding packets, intake sheets, benefits and government forms, contract templates. They share the same layout and boilerplate every time; only the filled-in values change. When one of those forms is emailed to the wrong person, shared "anyone with the link," or dropped into a Teams chat, generic content detectors often miss it — the *form itself* is the sensitive artifact, and a keyword or credit-card pattern isn't what identifies it. The leak is invisible until it becomes an incident.

The solution

Use **Microsoft Purview Document Fingerprinting** to turn each standard form into a **sensitive information type** built from the form's own text. Purview reads a blank/representative template, computes a compact **fingerprint** of its word pattern (the document itself is not stored), and thereafter recognizes *any* copy of that form — regardless of the values typed into it. That fingerprint is then wired into **Data Loss Prevention (DLP)** so movement of the forms through **email, SharePoint, OneDrive, and Teams** is detected and controlled — using only the Microsoft Purview portal and Security & Compliance PowerShell. As with every solution in this category, the DLP policy carrying the fingerprint is **proven in SIMULATION first** — Microsoft's built-in test mode that shows what *would* be caught, with zero user impact — then enforced to a **pilot ring**, then expanded.

What it delivers

- **Fingerprint SITs** for your standard forms — high-precision detection that keys off the form's structure, not a fragile keyword, with related forms grouped where sensible.
- A **DLP policy** across email, cloud, and Teams that detects the forms, proven in simulation, tuned for specificity, and enforced through policy tips, user justification/override, or block.
- A **measured false-positive rate** kept at/under an agreed threshold — the difference between a control people trust and one they ignore.
- An **incident/alert triage workflow** so every match is acted on, with an evidence trail in Activity Explorer and optional SIEM forwarding.
- A **fingerprint refresh process** so detection stays current as forms are revised.
- **Five PowerShell tools** (posture discovery, fingerprint/SIT builder, wiring validator, detection report, scope helper) — read-only or `-WhatIf` by default, no credentials in code.

Business outcomes

- **Standard forms detected** as they move through the channels data actually leaks through.

- **High precision, minimal disruption** — a fingerprint keys off the whole form pattern, so it fires on your forms and stays quiet on unrelated content; simulation-first plus instant rollback keeps the business running.
- **Operable and auditable** — matches raise alerts that are triaged to disposition, with evidence retained.
- **Self-guided at a fraction of consulting cost** — the same method a specialist would use, documented and tooled to run in-house.

Boundaries (so scope is clear)

Fingerprinting detects **text-pattern** matches — it will not catch an image-only scan, a password-protected file, or a form edited beyond recognition, and DLP locations/actions are licensing-dependent (**verify your tenant**). This solution provides the **technical configuration and the evidence**; *which* forms are in scope, *which* legal/regulatory obligation applies to each, and the **enforced action per form** are **compliance, privacy, and legal determinations**, captured alongside the delivered evidence.

1. Solution Overview — Document Fingerprinting (Microsoft Purview)

What this solution is

Most organizations run their sensitive business on **standard forms and templates** — a patent application form, an employee onboarding packet, a customer intake sheet, a government form, a contract template, a benefits enrollment form. These documents follow the *same layout and boilerplate text* every time, only the filled-in values change. That structural sameness is exactly what makes them detectable — and exactly what generic pattern detectors miss.

Microsoft Purview Document Fingerprinting converts a standard form into a **sensitive information type (SIT)** built from the form's own text. Purview reads the blank/representative template, builds a compact **fingerprint** of its word pattern, and thereafter recognizes *any document that carries that same pattern* — regardless of the data typed into it. You then wire that fingerprint SIT into **Data Loss Prevention (DLP)** so email, SharePoint, OneDrive, and Teams movement of those forms is detected and controlled.

This solution is a repeatable, evidence-driven way to fingerprint your standard forms and wire them to DLP using only the **Microsoft Purview portal** and **Security & Compliance PowerShell** (`Connect-IPPSSession`) — no third-party tooling. Its defining discipline, like every solution in this category, is that **the DLP policy carrying the fingerprint is built and proven in SIMULATION mode first** (Microsoft's built-in test mode that evaluates a policy against real traffic and shows what *would* have matched, with zero user impact), tuned, and only then enforced.

> ***Scope boundary.*** *This solution delivers the* ***technical method and evidence*** *for fingerprinting forms and detecting them with DLP. *Which* forms are sensitive, *which* legal/regulatory obligation applies to a given form, and *what* the enforced action must be are* ***business, compliance, privacy, and***

> ***legal determinations*** *— deferred to those functions. The solution helps you build and prove the control; it does not make the policy decision.*

Why this matters

• **Standard forms leak silently.** A completed intake form emailed to the wrong recipient, a blank

template with a filled record shared "anyone with the link," a benefits form dropped in a Teams chat — none of these necessarily contain a neat credit-card or SSN pattern, so a generic SIT never fires. The *form itself* is the sensitive artifact, and fingerprinting is how you detect it.

• **Fingerprinting beats keyword/regex for structured documents.** A pattern detector matches a string;

a fingerprint matches the *document's word structure*. For a form whose boilerplate is identical every time, a fingerprint gives high-precision detection that keyword rules can't match without heavy tuning.

• **It reuses the DLP you already run.** A fingerprint is just another SIT — once created it plugs into

the same DLP policies, simulation mode, policy tips, and Activity Explorer evidence trail as the rest of your DLP estate. No new control plane to operate.

• **Precision protects adoption.** Because a fingerprint keys off the whole form pattern rather than a

single number, it produces fewer false positives on unrelated content — which is what keeps a DLP control trusted rather than switched off.

How Document Fingerprinting works

1. **Pick the source template.** Choose a **blank or representative** copy of the form — text-based

(`.docx`, `.dotx`, `.pdf`, `.txt`, and similar supported formats). The template should contain the form's boilerplate text; the fingerprint is built from that text, not from filled-in values.

1. **Create the fingerprint.** Purview reads the file and computes a fingerprint — a small,

non-reversible representation of the document's word pattern. It does **not** store the document itself. In PowerShell this is `New-DlpFingerprint`; in the portal it is **Data classification** → **Classifiers** → **Sensitive info types** → **Create** → **Document fingerprint** (verify availability in your tenant).

1. **Wrap the fingerprint in a sensitive information type.** One or more fingerprints become a named

fingerprint SIT (`New-DlpSensitiveInformationType -Fingerprints ...`). One SIT can carry several related form fingerprints (e.g. all HR onboarding forms).

1. **Reference the SIT in a DLP policy** exactly like any other SIT — condition *Content contains* the

fingerprint SIT, across the chosen locations, with an action (audit / policy tip / restrict / block).

1. **Detection at runtime.** When a document whose word pattern matches the fingerprint moves through a

covered channel, the policy matches — even though the *values* in the form differ from the template.

1. **Evidence & handling.** Matches raise DLP **alerts** and flow to **Activity Explorer**, feeding the

same triage workflow as the rest of your DLP program.

What a fingerprint can and cannot detect (be honest with buyers)

• **Detects:** documents that carry substantially the **same text pattern** as the fingerprinted

template — the filled-in versions of your standard forms, forwarded copies, and re-saved variants that keep the boilerplate.

• **Does not reliably detect:** a form that is **only an image / scanned** (no extractable text), a

password-protected file, a document that contains **only a small fragment** of the form's text, or a form so heavily edited that the boilerplate no longer resembles the template. Fingerprinting is a **text-pattern** match — no extractable text, no match.

• **Format & size limits apply** (supported file types and a maximum file size per Microsoft's

documentation) — **verify current limits for your tenant** before fingerprinting large or unusual files.

• Fingerprinting is **one detection technique** among several (built-in SITs, custom SITs, EDM,

trainable classifiers). Use it where the *form structure* is the signal; use the others where a value pattern or a learned category is the signal.

Naming convention (used in every doc and script)

Every artifact this solution creates uses `<Purpose> – <Scope> – <Stage>`, reused verbatim across the docs, scripts, and the Purview portal. The standard set:

Artifact	Name
Fingerprint (single form)	`Onboarding Form – Fingerprint – Source`
Fingerprint SIT (one or more forms)	`Standard Forms – Fingerprint SIT – Classification`
DLP policy (simulation)	`DLP – Fingerprinted Forms – Simulation`
DLP rule	`Rule – Fingerprinted Forms – ContentContains`
Enforced pilot policy scope	`DLP – Fingerprinted Forms – Enforce Pilot`
Pilot user ring	`Fingerprint Pilot Ring – Stage1`
Policy-tip / override text	`Fingerprint Notice – Standard – AllPolicies`
Incident/alert workflow	`Fingerprint Alerts – Triage – Operations`

The standard deployment model

> ***Assess*** *which forms are candidates and confirm licensing/roles →* ***Design*** *the fingerprint SIT(s) and the DLP policy →* ***Configure*** *the fingerprints and build the policy* ***in simulation*** *→* ***Pilot*** *(simulate → measure → enforce to a ring) →* ***Validate*** *against pass/fail gates (forms detected, false positives low, incidents handled) →* ***Expand*** *to full-scope enforcement and ongoing operation. Every high-impact change is piloted in simulation first, with a documented rollback. The DLP policy is never switched*

straight to enforce.

2. Requirements — Document Fingerprinting (Microsoft Purview)

These are the **solution-specific** prerequisites for fingerprinting standard forms and wiring them to DLP. They build on — and do not repeat — the six shared prerequisites in `global-requirements-purview.md` (tenant + compliance licensing, least-privilege Purview roles, S&C PowerShell + Graph toolchain, in-scope data sources, compliance/records/privacy/legal stakeholders, and a pilot scope + change control). Verify each item below before starting the Implementation Guide.

For every requirement: **Description** (what it is), **Why required**, **How to verify**, **How to implement if missing**.

2.1 Licensing sufficient for DLP and document fingerprinting

• **Description:** Document fingerprinting produces a **sensitive information type** consumed by **Purview DLP**. Fingerprinting has long been available as part of Exchange/DLP, but the DLP **locations** and **actions** you enforce (e.g. endpoint DLP, advanced conditions) are **plan-dependent** — several are **Microsoft 365 E5 / E5 Compliance**, and some capabilities are in **preview**. This extends GR-#1 with the fingerprint/DLP-specific plan detail.

• **Why required:** A fingerprint SIT is only useful once a DLP policy in a licensed location references it. A location or action your plan doesn't include will not function.

• **How to verify:** In the **Microsoft 365 admin center** confirm the subscription/add-ons; in the **Purview portal** → **Data loss prevention** and **Data classification** → **Classifiers** → **Sensitive info types** confirm you can create a **Document fingerprint** SIT and target the locations you intend to use. **Flag "verify your licensing/tenant"** — availability evolves.

• **How to implement if missing:** Acquire/assign the required licensing before building policies in gated locations; scope the initial rollout to features your plan covers (email/SharePoint/OneDrive/ Teams DLP with a fingerprint SIT is the baseline).

2.2 Fingerprint/DLP-specific least-privilege roles

• **Description:** Creating fingerprint SITs and DLP policies requires the **Purview DLP role groups** — **Compliance Administrator** or **Compliance Data Administrator** to create/manage SITs and DLP policies, **DLP Compliance Management** for policy management, **View-Only DLP Compliance Management** for read-only review, and the **Data Loss Prevention** role for alert/incident handling. The fingerprint cmdlets (`New-DlpFingerprint`, `New-DlpSensitiveInformationType`) run in **Security & Compliance PowerShell** and require the matching role. Global Administrator only where genuinely required. Extends GR-#2.

• **Why required:** Purview enforces fine-grained role groups; SIT/DLP configuration and alert triage

require the matching role, and least privilege keeps the blast radius small.

- **How to verify: Purview portal** → **Settings** → **Roles & scopes** → **Permissions** — confirm the account's

role-group membership; confirm `New-DlpSensitiveInformationType` is available in your S&C PowerShell session.

- **How to implement if missing:** Assign the least-privilege DLP role group under change control;

separate SIT/policy authors from alert triagers where possible.

2.3 The source form templates to fingerprint (blank/representative, text-based)

- **Description:** For each form you will protect, a **blank or representative copy** of the template in

a **supported, text-based format** (`.docx`, `.dotx`, `.pdf`, `.txt` and similar) whose **boilerplate text is intact** — the fingerprint is built from that text. Scanned/image-only forms have no extractable text and cannot be fingerprinted.

- **Why required:** The fingerprint *is* the template's word pattern. A poor source (image-only,

password-protected, or an already-filled copy with little boilerplate) yields a weak or unusable fingerprint.

- **How to verify:** Confirm you hold the current version of each form as a text-based file; open it and

confirm the boilerplate text is selectable (not a flattened image). Confirm the file is within Microsoft's supported **type and size** limits (**verify current limits for your tenant**).

- **How to implement if missing:** Obtain the authoritative blank template from the form owner; if only

a scan exists, source or recreate a text-based version, or OCR it to extractable text before fingerprinting.

2.4 A form inventory and classification intent

- **Description:** A documented list of the **standard forms in scope** (name, owner, version,

sensitivity, the workloads they travel through) mapped to the **fingerprint SIT** that will detect each — grouped where several related forms belong in one SIT. Extends GR-#4.

- **Why required:** Fingerprinting is per-form; without knowing *which* forms matter, who owns them, and

how they move, policy scoping and SIT grouping are guesswork.

- **How to verify:** Confirm the form list, owners, and versions with the business/compliance

stakeholders; the Discovery script (#1) reports existing fingerprint SITs and DLP coverage as a starting baseline.

- **How to implement if missing:** Build the form inventory with the form owners before authoring

fingerprints; agree which forms group into which SIT.

2.5 A form-version refresh process

• **Description:** A named owner and cadence for **re-fingerprinting when a form's template changes** —

a fingerprint reflects the template *as fingerprinted*; a materially revised form drifts from its fingerprint and detection degrades.

• **Why required:** Standard forms are revised. Without a refresh process, detection silently decays as the live forms diverge from the fingerprinted template — a false sense of coverage.

• **How to verify:** Confirm the form owner will notify on template changes and that a refresh step (add the new template's fingerprint to the SIT, retire the old) is agreed.

• **How to implement if missing:** Define the refresh trigger and owner; a fingerprint SIT can hold **multiple fingerprints**, so a revised form is added rather than replaced wholesale.

2.6 A pilot ring, simulation dwell time, and change control (DLP-specific)

• **Description:** A named **pilot user ring**, an agreed **simulation dwell time** (how long the policy runs in simulation before enforcement — long enough to capture representative form traffic), and the **change-control window** for moving simulation → enforce. Extends GR-#6.

• **Why required:** DLP is high-impact; enforcing a form-detection policy without a simulation baseline and a limited ring risks blocking legitimate business. The house model mandates **simulation first**, then a ring, then expand.

• **How to verify:** Confirm the pilot ring membership, that **simulation mode** is selected on the new policy, and the approval path to enforce.

• **How to implement if missing:** Define the ring and dwell time and agree change control before enabling any enforcement action.

2.7 An incident/alert triage owner and destination

• **Description:** A named owner (or queue) for **DLP alerts** raised on fingerprint matches (**Purview** → **Data loss prevention** → **Alerts**), plus where matches are reviewed (**Activity Explorer**) and, optionally, forwarded (SIEM/Microsoft Sentinel). Severity and response actions defined.

• **Why required:** A form match with no one to triage it is not a control. Someone must own triage, severity, and response.

• **How to verify:** Confirm the triage owner/queue, the alert-severity scheme, and any SIEM forwarding.

• **How to implement if missing:** Assign the owner, define severity/response, and (optionally) wire SIEM forwarding before enforcement generates real alerts.

Requirements summary

#	Requirement	Gate
GR 1–6	Category prerequisites (`global-requirements-purview.md`)	Tenant/licensing, roles, toolchain, scope, stakeholders, pilot + change control
2.1	DLP + fingerprint licensing (verify tenant)	SIT creation + target locations available
2.2	DLP/fingerprint least-privilege roles	SIT/policy authoring + triage roles assigned
2.3	Source form templates (text-based, blank)	Fingerprintable input in hand
2.4	Form inventory + classification intent	Policies key off the right forms/SITs
2.5	Form-version refresh process	Detection stays current as forms change
2.6	Pilot ring + simulation dwell + change control	Simulate-first enforced
2.7	Alert/incident triage owner + destination	Matches are acted on

> ***Deferred to compliance/records/privacy/legal:*** *which forms are in scope, which legal/regulatory obligation applies to each, and the enforced action per form. This solution provides the technical configuration and evidence, not the determination.*

Global Requirements — Microsoft Purview Solutions

These requirements apply to **every** solution in the **Purview** category (implementing and managing Microsoft Purview, sensitivity & retention labels, DLP, data lifecycle & records, insider risk, communication compliance, audit, data security posture management, and data governance). Verify each before starting a solution's implementation steps.

> ***Vendor boundary (applies to every Purview solution).*** *The **Microsoft Purview portal** (purview.microsoft.com), the **Microsoft 365 admin center**, **Security & Compliance PowerShell** (`Connect-IPPSSession`), and **Microsoft Graph** are authored at exact, current click-by-click / cmdlet detail from Microsoft's public documentation. Microsoft Purview features are heavily **licensing-/plan-dependent** (many require **Microsoft 365 E5** or the **E5 Compliance** add-on; some are in preview) and the portal is **actively evolving** — capability availability and portal paths are **cited or flagged "verify your licensing/tenant"**, never asserted or invented. **Regulatory, records-retention, and legal determinations** (retention schedules, records file plans, legal holds, insider-risk case actions, regulatory applicability) are **deferred to the organization's compliance, records, privacy, and legal functions** — the solutions provide the technical configuration method and evidence, not the determination. Automation is **PowerShell** (Security & Compliance PowerShell / Graph) — never with hardcoded credentials or secrets.*

1. Microsoft 365 tenant with the required Purview/compliance licensing

- **Description:** A Microsoft 365 tenant with the **compliance licensing** the capability requires

(E3 for baseline; **E5 / E5 Compliance** for advanced DLP, auto-labeling, insider risk, communication compliance, Audit Premium, DSPM, etc.).

- **Why required:** Most Purview capabilities are plan-gated; the feature won't be available without the

right plan.

- **How to verify:** In the M365 admin center confirm the subscription/add-ons; check the feature is

visible in the Purview portal.

- **How to implement if missing:** Assign/purchase the required licensing before starting.

2. Purview roles (least-privilege) and portal access

- **Description:** Access with the least-privilege **Purview / compliance role** the solution names

(e.g. Compliance Administrator, Compliance Data Administrator, Information Protection, Records Management, Insider Risk, eDiscovery, Audit roles), with Global Administrator only where required.

- **Why required:** Purview uses fine-grained role groups; configuration requires the matching role.
- **How to verify:** Purview portal → **Roles & scopes / Permissions** — confirm the role membership.
- **How to implement if missing:** Assign the least-privilege role group under change control.

3. Security & Compliance PowerShell + Graph toolchain

- **Description:** A workstation with **Security & Compliance PowerShell** (`Connect-IPPSSession` via the

ExchangeOnlineManagement module) and **Microsoft Graph PowerShell**, modern-auth / MFA-capable admin.

- **Why required:** Configuration, automation, and the provided scripts run through these.
- **How to verify:** `Connect-IPPSSession` and `Connect-MgGraph` succeed with the admin account.
- **How to implement if missing:** Install the modules and confirm modern-auth sign-in before running steps.

4. Data sources & scope in scope

- **Description:** The **workloads and data sources** in scope (Exchange, SharePoint, OneDrive, Teams,

endpoints/devices, and — for data governance — the broader data estate) and a **data-classification / sensitivity** intent to drive labels, DLP, and retention.

- **Why required:** Purview policies key off the in-scope workloads and the classification scheme.
- **How to verify:** Confirm the in-scope workloads and the classification/sensitivity approach with

stakeholders.

- **How to implement if missing:** Define the scope and classification intent before authoring policies.

5. Compliance, records, privacy & legal stakeholders

• **Description:** Engagement with the **compliance, records-management, privacy, security, and legal**

stakeholders whose policies the configuration must satisfy (retention schedules, records file plans, regulatory obligations, insider-risk and communication-compliance policy, legal hold).

• **Why required:** Purview configuration must align to enterprise policy and legal/records requirements,

not technical defaults; several capabilities have HR/legal sensitivity.

• **How to verify:** Confirm the applicable policies, the records schedule, and named approvers.

• **How to implement if missing:** Identify the owners and capture the requirements before configuring.

6. A pilot scope and change control

• **Description:** A **pilot scope** (users/sites/policies in simulation or a limited ring) to validate

on before tenant-wide enforcement, and a **change-control** process — most Purview policies support a **simulation / test / disabled** mode that MUST be used before enforcement.

• **Why required:** Every solution follows an **Assess → Design → Configure → Pilot (simulate) → Validate

→ Expand (enforce)** model; DLP, labels, retention, and insider-risk changes are high-impact.

• **How to verify:** Confirm a named pilot scope, that simulation/test mode is available, and a change

window/approval path.

• **How to implement if missing:** Define the pilot and agree change control before enforcing anything.

*The deployment model across every Purview solution is the same: **Assess** → **Design** → **Configure** → **Pilot (simulate/test)** → **Validate** → **Expand (enforce)**, with explicit pass/fail gates and a documented rollback. Automation is delivered as **PowerShell** (Security & Compliance PowerShell / Graph) — never with hardcoded credentials or secrets. Regulatory/records/legal determinations are deferred to the relevant governance functions.*

3. Implementation Guide — Document Fingerprinting (Microsoft Purview)

This is the exact, ordered procedure to fingerprint standard forms and wire them to DLP — the DLP policy carrying the fingerprint built in **SIMULATION** first, measured, then enforced to a ring, then expanded. Portal paths and cmdlets are from Microsoft's public documentation; **licensing-/plan-dependent features and evolving portal paths are flagged "verify your licensing/tenant"** and are never asserted for your tenant. Do the phases in order. Each phase names its artifacts (from the Overview naming table), states a **pass/fail gate**, and gives a **rollback**.

> ***Automation.*** *The five delivered PowerShell tools run through **Security & Compliance PowerShell** (`Connect-IPPSSession`). They are **read-only or** `-WhatIf` **by default**; writes require an explicit* `-Commit` *switch. No credentials are stored in code.*

> ***Determinations deferred.*** **Which* forms are in scope, *which* legal/regulatory obligation applies to each, and the *enforced action* per form are* ***compliance/privacy/legal*** *decisions. This guide configures and proves the control; capture those decisions before Phase 4.*

Phase 0 — Connect and baseline (Assess)

1. **Connect the toolchain** (GR-#3). On the admin workstation:

```
Install-Module ExchangeOnlineManagement -Scope CurrentUser -Force   # provides Connect-IPPSSession
Connect-IPPSSession -UserPrincipalName admin@contoso.com            # Security & Compliance PowerShell
```

1. **Run the posture assessment** (delivered script #1, read-only):

```
.\CLSFINGER-Discovery.ps1 -CsvPath .\CLSFINGER-Posture.csv
```

It reports existing **fingerprint SITs**, the DLP policies that reference them and their **mode**, and readiness signals.

1. **Confirm licensing and roles** (Req 2.1, 2.2): confirm you can create a **Document fingerprint** SIT

and target the DLP locations you intend to use, and that your account holds the DLP roles.

Pass/fail gate: you have a written baseline (existing fingerprint SITs + any DLP referencing them), confirmed licensing, and the roles to author SITs and policies. **Rollback:** none (read-only).

Phase 1 — Assemble and validate the source forms (Design)

Decide, per in-scope form, that fingerprinting is the right technique (the *form structure* is the signal). Do **not** fingerprint image-only or password-protected files — they have no extractable text.

1. **Build the form inventory** (Req 2.4): for each form, record name, owner, version, sensitivity, and

the workloads it travels through. Group related forms (e.g. all HR onboarding forms) into one planned fingerprint SIT where it makes sense.

1. **Collect a blank/representative template per form** (Req 2.3) in a supported text-based format

(`.docx`, `.dotx`, `.pdf`, `.txt`, ...). Open each and confirm the **boilerplate text is selectable** (not a flattened image), and that the file is within Microsoft's supported **type/size** limits (**verify current limits for your tenant**).

1. **Name each source and its SIT** from the Overview table — e.g. source `Onboarding Form – Fingerprint

– Source`, `SIT` Standard Forms – Fingerprint SIT – Classification`.

Pass/fail gate: every in-scope form has a text-based template in hand, is confirmed fingerprintable, and is assigned to a named planned SIT. **Rollback:** none (no tenant change yet).

Phase 2 — Create the fingerprints and the SIT (Configure)

Create the fingerprint(s), then wrap them in a named SIT. **Portal path:** *Purview portal → Data classification → Classifiers → Sensitive info types → Create sensitive info type → Document fingerprint* (verify availability in your tenant). Equivalent PowerShell (used by the delivered script #2):

1. **Create a fingerprint from each template** with `New-DlpFingerprint`. In PowerShell 5.1:

```
$formBytes = Get-Content ".\Onboarding-Template.docx" -Encoding Byte -ReadCount 0
$fp = New-DlpFingerprint -FileData $formBytes -Description "Onboarding Form – Fingerprint – Source"
```

(PowerShell 7 replaces `-Encoding Byte` with `-AsByteStream`; the delivered script #2 handles both.) `New-DlpFingerprint` computes the fingerprint object; **the document itself is not stored.**

1. **Wrap one or more fingerprints in a SIT** with `New-DlpSensitiveInformationType`:

```
New-DlpSensitiveInformationType -Name "Standard Forms – Fingerprint SIT – Classification" `
  -Fingerprints $fp -Description "Detects our standard onboarding/intake forms"
```

To add more forms into the same SIT, pass an array: `-Fingerprints @($fp1,$fp2,$fp3)`.

1. **Do it safely with the delivered helper** (script #2, `-WhatIf` by default):

```
.\CLSFINGER-CreateFingerprint.ps1 -TemplatePath .\Onboarding-Template.docx `
  -SitName "Standard Forms – Fingerprint SIT – Classification" -WhatIf
```

Run with `-Commit` only to actually create/update the SIT. The script never touches a DLP policy's enforce state.

1. **Confirm the SIT exists**: `Get-DlpSensitiveInformationType -Identity "Standard Forms – Fingerprint SIT – Classification"` (or script #1). Note its **GUID/Publisher** for reference.

Pass/fail gate: the fingerprint SIT exists, carries the intended form fingerprint(s), and is visible in **Data classification** → **Classifiers** → **Sensitive info types**. **Rollback:** `Remove-DlpSensitiveInformationType -Identity "..."` — a SIT not yet referenced by any policy has no runtime effect, so removal is clean.

Phase 3 — Build the DLP policy in SIMULATION (Configure — no user impact)

Reference the fingerprint SIT from a DLP policy in **simulation mode**. Simulation evaluates the policy against real traffic and records what *would* match, with **no** enforcement.

1. **Create the policy —** `DLP – Fingerprinted Forms – Simulation`**.**

Portal: **Data loss prevention** → **Policies** → **Create policy** → *Custom* → **Locations:** start with **Exchange email, SharePoint sites, OneDrive accounts, Teams chat and channel messages** (add **Devices/Endpoint** only if licensed and onboarded) → **Rule** `Rule – Fingerprinted Forms – ContentContains`**:** condition *Content contains → Sensitive info types →* select `Standard Forms – Fingerprint SIT – Classification` → action *Restrict access or encrypt* (kept inert by simulation) → **Policy mode: Run the policy in simulation mode** (optionally *with policy tips*). Finish.

1. **Set the user experience** (applies once enforced): define `Fingerprint Notice – Standard – AllPolicies` policy-tip text and enable **allow override with justification** where compliance permits.

1. **Verify the wiring** (delivered script #3, read-only):

```
.\CLSFINGER-Validation.ps1 -SitName "Standard Forms – Fingerprint SIT – Classification"
```

It confirms the SIT exists, that `DLP – Fingerprinted Forms – Simulation` references it, and that the policy is in simulation mode.

Pass/fail gate: the policy exists, is confirmed in **simulation**, references the fingerprint SIT, and is scoped to the correct locations. **Rollback:** delete the simulation policy — simulation has no user impact, so removal is clean.

Phase 4 — Measure and confirm detection (Pilot — simulate)

Let the policy run in simulation for the agreed **dwell time** (Req 2.6) to capture representative form traffic, and prove the fingerprint actually detects the live forms.

1. **Positive detection test.** Take a **filled** copy of a fingerprinted form (with **clearly-fake**

data — never real personal data) and move it through a covered channel (e.g. email it internally). Confirm it appears as a **simulated match**.

1. **Review what matched. Purview portal** → **Data loss prevention** → **Alerts** (simulation) and **Data

classification → Activity explorer** (filter to DLP) show every simulated match — file/message, SIT, action. Export with the reporting script:

```
.\CLSFINGER-Report.ps1 -Days 14 -CsvPath .\CLSFINGER-Matches.csv
```

1. **Classify each match** as true positive or **false positive**. Fingerprints are high-precision, but

watch for over-broad matches where the fingerprinted boilerplate also appears in unrelated documents (e.g. a common letterhead). If false positives appear:

- **Re-source a cleaner template** (more form-specific boilerplate, less generic letterhead).
- **Split** an over-broad SIT so each form is detected on its own distinctive text.
- Add a targeted **exception** where justified (specific sender/site), narrowly and documented.

1. **Confirm the negative case.** Move an **unrelated** document through the same channel and confirm it

is **not** matched — verifying the fingerprint is specific to the form.

Pass/fail gate: the filled form is detected in simulation; unrelated content is not; false positives are understood and at/under the agreed threshold. **Rollback:** re-source/re-create the fingerprint (Phase 2) or delete the simulation policy — no user impact.

Phase 5 — Enforce in a pilot ring, then expand (Pilot → Enforce)

1. **Scope to the pilot ring.** Add `Fingerprint Pilot Ring – Stage1` as the **included** scope on the

policy (all other users excluded for now). The delivered scope helper does this safely:

```
.\CLSFINGER-Set-Scope.ps1 -PolicyName "DLP – Fingerprinted Forms – Simulation" `
  -IncludeGroup "Fingerprint Pilot Ring – Stage1" -WhatIf
```

Run with `-Commit` to apply.

1. **Move the ring to enforcement.** Change **Policy mode** from *simulation* to **Turn it on / Enforce**

(rename the intent to `DLP – Fingerprinted Forms – Enforce Pilot`). Start with **policy tips + allow-override (justification)** rather than hard block, so the pilot can report issues while keeping a business escape hatch.

1. **Confirm enforcement is live for the ring only** (script #3 reports mode + scope). Confirm a ring user sees the `Fingerprint Notice – Standard – AllPolicies` tip on a test match.

1. **Wire incident handling.** Set up `Fingerprint Alerts – Triage – Operations`: assign the alert owner/queue (Req 2.7), set severities, and (optionally) forward to SIEM/Sentinel. Triage a real pilot alert end-to-end to prove the workflow.

1. **Expand and harden.** Once the pilot gate passes, expand scope from the ring to the full intended population, keeping documented exclusions; **escalate the action** per compliance's determination (e.g. tip+override → **block**) one form/location at a time, re-validating after each. Schedule the reporting script for a recurring review, and **refresh fingerprints** when a form template changes (Req 2.5) by adding the new template's fingerprint to the SIT.

Pass/fail gate: the ring is enforced (tips shown, overrides captured), matches raise alerts, the triage workflow produces a dispositioned incident, and scope then expands with the compliance-mandated action per form. No enforcement outside the ring during pilot. **Rollback:** set the policy back to **simulation mode** (single setting) — instantly removes enforcement with no data change; the policy or SIT can be disabled/removed without affecting other DLP policies.

Order-of-operations summary

Phase	Do	Artifact(s)	Gate	Rollback
0 Assess	Connect, baseline, confirm licensing/roles	Posture CSV	Baseline + readiness confirmed	n/a (read-only)
1 Design	Inventory forms, collect text templates	Form inventory	Every form → fingerprintable template	none (no change)
2 Configure	Create fingerprint(s) + SIT	Fingerprint SIT	SIT exists with form fingerprints	Remove SIT
3 Configure	Build DLP policy in **simulation**	Sim policy + rule	In simulation, references SIT, scoped	Delete sim policy
4 Pilot(sim)	Prove detection, measure FPs	Matches CSV	Form detected, unrelated not, FP ≤ threshold	Re-fingerprint / delete policy
5 Enforce	Ring → tips/override → expand → block	Pilot ring, alert triage	Ring enforced, incident dispositioned, expanded	Back to simulation

5. Testing & Validation — Document Fingerprinting (Microsoft Purview)

Validate the fingerprinting implementation against explicit **pass/fail gates**. The three headline gates are: **(A) the forms are detected**, **(B) the false-positive rate is low / detection is specific**, and **(C) incidents are handled**. Run every test with **clearly-fake sample data** — fill the forms with obviously non-real values (e.g. `John Doe`, `000-00-0000`), never real personal data — and record the evidence.

> *All tests assume the fingerprint SIT and DLP policy from the Implementation Guide, first proven in **simulation** and then enforced to the **pilot ring**. Endpoint tests (if the Devices location is used) require an **onboarded** device.*

Test A — Detection (the forms are detected)

#	Test	Steps	Pass condition
A1	Filled form detected in email	Fill a fingerprinted form with fake data, save it, and email it to another user covered by the policy.	The document matches the fingerprint SIT; a DLP match/alert (simulation or enforced) is raised for the correct policy.
A2	Form on SharePoint/OneDrive	Upload a filled fingerprinted form to a covered site/account (and, in enforce, attempt an external share).	Match recorded in Activity Explorer; in enforce, external share is restricted per the action.
A3	Form in Teams	Share a filled fingerprinted form in a covered Teams chat/channel.	Match recorded; in enforce, tipped/blocked per the action.
A4	Re-saved / forwarded variant	Take the filled form, re-save it (same boilerplate) and move it again.	Still detected — confirms the fingerprint keys off the form's word pattern, not the exact file.
A5	Endpoint (if Devices used)	On an onboarded ring device, copy the filled form to removable media / print it.	Activity audited/restricted per policy; endpoint DLP event appears in Activity Explorer.
A6	Multi-form SIT	If the SIT carries several form fingerprints, test one filled copy of **each** form.	Each distinct form is detected by the same SIT.

Gate A (pass): the filled forms (A1–A4, and A5 if endpoint used) are detected across the covered channels and, in enforce, produce the configured action and a logged event.

Test B — Specificity / false-positive rate (adoption gate)

#	Test	Steps	Pass condition
B1	Unrelated content negative	Move representative **unrelated** documents (memos, ordinary attachments — no form boilerplate) through the covered channels.	Unrelated content is **not** matched — no spurious tips/blocks.
B2	Simulation FP baseline	Over the agreed dwell time, export matches (`CLSFINGER-Report.ps1 -Days <n>`) and classify each as TP/FP.	**False-positive rate** ≤ **the agreed threshold** (define with compliance).
B3	Shared-boilerplate check	If forms share a common letterhead/header, move a **different** document that also carries that letterhead.	It is **not** matched by the form's fingerprint — confirms the fingerprint keys off form-specific text, not just the common header. If it matches, re-source a more form-specific template (Impl. Phase 4).
B4	Fragment / partial negative	Move a document containing only a **small fragment** of the form's text.	Correctly **not** matched (or matched only per the intended sensitivity) — confirms the fingerprint requires substantial form structure.
B5	Non-fingerprintable input	Attempt with an **image-only** scan of the form and a **password-protected** copy.	Correctly **not** detected — documents the known limitation so operators aren't surprised.

Gate B (pass): unrelated/partial/non-text content is not matched; measured FP rate on the simulation sample is at/under threshold with true positives intact.

Test C — Incident / alert handling (the control is operable)

#	Test	Steps	Pass condition
C1	Alert raised	Trigger a match (Test A1).	A DLP **alert** appears in **Purview** → **Data loss prevention** → **Alerts** with the correct policy, severity, and evidence.
C2	Triage workflow	The `Fingerprint Alerts – Triage – Operations` owner/queue picks up the alert, sets status, and dispositions it.	Alert moves through the defined states to a resolved disposition by the named owner.
C3	Activity Explorer trail	Locate the same event in **Data classification** → **Activity explorer**.	Full evidence trail present (user, location, SIT, action) for investigation.
C4	SIEM forwarding (if configured)	Confirm the event reaches the SIEM/Sentinel destination (Req 2.7).	Event present downstream with usable fields.

Gate C (pass): matches raise alerts, alerts are triaged to disposition by the owner, and the evidence trail (and SIEM forward, if used) is present.

Simulation-first & rollback validation

#	Test	Pass condition
S1	Nothing enforced outside the ring	Users outside `Fingerprint Pilot Ring – Stage1` experience no blocking during pilot.
S2	Instant rollback	Setting the enforced policy back to **simulation mode** immediately removes enforcement with no data change (verify a previously-blocked action now only simulates).
S3	Scripts are safe-by-default	`CLSFINGER-CreateFingerprint.ps1` and `CLSFINGER-Set-Scope.ps1` run **without** `-Commit` make no change (`-WhatIf` output only); Discovery/Validation/Report scripts are read-only.
S4	Refresh path	After adding a revised form's fingerprint to the SIT (Req 2.5), re-run Test A on the new template and confirm it is detected.

Evidence to capture (for the deliverable record)

- Baseline posture CSV (Phase 0) and the form inventory.
- Confirmation the fingerprint SIT exists and is referenced by the simulation policy (Validation script).
- Simulation match export + TP/FP classification and the computed **false-positive rate** (Gate B).
- Screenshots/log of Test A detection and the raised alerts (Gate A/C).
- The dispositioned incident from Test C2 and, if used, the SIEM forward (C4).
- Confirmation outside-ring users were unaffected (S1) and that simulation rollback works (S2).

> ***Determination boundary.*** *These tests prove the **control works** — that the forms are detected and handled. Whether a given form's enforced action and FP threshold are *acceptable* is a compliance/privacy/legal sign-off, captured alongside the evidence above.*

6. Troubleshooting — Document Fingerprinting (Microsoft Purview)

Symptom-first fixes for the failures you actually hit fingerprinting standard forms and running them in DLP. Most problems are one of: a fingerprint that won't build, a form that isn't detected, a fingerprint that over-matches, no alert on a match, or detection drifting as a form is revised.

Creating the fingerprint / SIT

`New-DlpFingerprint` **fails or produces a weak fingerprint.**

- Confirm the source is **text-based with selectable boilerplate**, not a flattened image or scan — a

fingerprint is built from extractable text, so an image-only file yields nothing usable. OCR the file to extractable text first, or source a text-based template.

- Confirm the file is a **supported type** and **within the size limit** (`.docx`, `.dotx`, `.pdf`,

`.txt`, and similar; **verify current supported types/limits for your tenant**). Password-protected files can't be read.

- On **PowerShell 7**, `Get-Content -Encoding Byte` is not valid — use `-AsByteStream`. The delivered

`CLSFINGER-CreateFingerprint.ps1` handles both PS 5.1 and PS 7 automatically.

`New-DlpSensitiveInformationType` **fails or the SIT doesn't appear.**

- Confirm your account holds **Compliance Administrator / Compliance Data Administrator** — a missing

role surfaces as "cmdlet not recognized" or access denied.

- **Propagation:** a newly created SIT can take time to appear in the portal (**Data classification →

Classifiers → Sensitive info types**) and to become selectable in a DLP rule. Wait and re-check with `Get-DlpSensitiveInformationType`.

Detection (form not matched)

A filled copy of a fingerprinted form isn't detected.

• Confirm the SIT is actually **referenced by a DLP policy** and the policy is in a **mode that

evaluates **— an *unreferenced* SIT is inert; a *disabled* policy does nothing; a** simulation** policy evaluates and logs but does not block. Check with `CLSFINGER-Validation.ps1` and `CLSFINGER-Discovery.ps1`.

• **Policy propagation takes time** — new/changed DLP policies can take **up to ~1 hour (longer for some

locations)** to take effect. Wait before concluding it's broken.

• Confirm the **location scope** includes where you're testing and the test identity isn't in an

exclusion (`CLSFINGER-Set-Scope.ps1` reports the Exchange include/exclude scope).

• The live form may have **drifted from the fingerprinted template** — if the boilerplate was

substantially revised, re-fingerprint the current template (Req 2.5 / Impl. Phase 5 refresh) and add it to the SIT.

• The moved document may carry **only a fragment** of the form's text — fingerprinting needs substantial

form structure to match; that a fragment isn't caught is expected behavior.

Matches appear in simulation but nothing is blocked.

• That is **correct** — simulation logs what *would* happen without enforcing. To block, move the policy

to **enforce** for the pilot ring (Impl. Phase 5). If you expected blocking, you're still in simulation.

Over-matching (false positives)

Unrelated documents are matching the fingerprint.

• The most common cause is **shared boilerplate** — the fingerprinted template's distinctive text is

mostly a **common letterhead/header** that also appears on unrelated documents. **Re-source** a copy with more form-specific boilerplate (trim the generic chrome) and re-fingerprint.

• If **one SIT groups several forms** and it over-fires, **split** it so each form is detected on its own

distinctive text.

• Add a targeted **exception** (specific sender/site) where justified — narrowly, with a documented

reason. Re-simulate after each change and re-measure with `CLSFINGER-Report.ps1`; don't stack changes blind.

Alerts / incident handling

Form matches occur but no alert appears / no one is notified.

• Confirm **alerts are configured** on the rule and check **Purview** → **Data loss prevention** → **Alerts**.

Some low-severity matches aggregate rather than alert per event.

• Confirm **audit logging is enabled** in the tenant — the report script and Activity Explorer rely on it.

• Confirm the **triage owner/queue** (`Fingerprint Alerts – Triage – Operations`) exists and, if

forwarding to SIEM/Sentinel, that the connector is healthy (Req 2.7).

The report script returns no matches.

• **Get-DlpDetectionsReport** may not be available in a pure S&C session — the script falls back to `Search-UnifiedAuditLog`; connect **Exchange Online** and confirm the **View-Only DLP / Audit** role.

• **Audit latency / dwell time:** DLP events take time to surface and a fresh policy may simply have no matches yet. Widen `-Days` and re-run.

Refresh / drift

Detection was working, now it's missing the current form.

• The form was likely **revised** — the fingerprint reflects the template *as fingerprinted*. Obtain the new template, run `CLSFINGER-CreateFingerprint.ps1 -Commit` to **add** its fingerprint to the same SIT (a SIT can hold multiple fingerprints), and retire the old template's fingerprint where appropriate.

Connectivity / tooling

`Connect-IPPSSession` **or a fingerprint/DLP cmdlet fails.**

• Ensure the **ExchangeOnlineManagement** module is installed/updated and you're using a **modern-auth / MFA-capable** admin.

• Confirm the account holds the required **DLP role** (Compliance Administrator / Compliance Data Administrator / DLP Compliance Management).

A script made an unexpected change.

• The delivered scripts are **read-only or** `-WhatIf` **by default**; writes require `-Commit`.

`CLSFINGER-CreateFingerprint.ps1` only ever creates/updates a SIT (never a policy), and `CLSFINGER-Set-Scope.ps1` **never changes a policy's enforce/simulation mode**. The safe recovery for enforcement is always to set the policy back to **simulation** in the portal (one setting, no data change).

Fast triage table

Symptom	First check
Fingerprint won't build	Text-based/selectable? supported type/size? PS7 → `-AsByteStream`
SIT won't create/appear	Role (Compliance Admin), propagation delay

Symptom	First check
Form not detected	SIT referenced? policy mode (disabled/sim)? propagation? scope/exclusion? template drift?
Nothing blocked	Still in **simulation** — expected; enforce the ring to block
Unrelated docs matching	Shared letterhead — re-source form-specific template; split SIT; add exception
No alert on match	Alert config on rule, audit logging on, triage owner, SIEM connector
Report returns nothing	Exchange Online connected? audit role/latency, widen `-Days`
Was detecting, now misses	Form revised — re-fingerprint current template into the SIT
Cmdlet/connect fails	Module updated, modern auth, correct DLP role

7. Best Practices — Document Fingerprinting (Microsoft Purview)

Field-tested practices for fingerprinting standard forms and running them in DLP so the control stays precise, trusted, and current. These complement the Implementation and Testing guides.

Fingerprint the right source

• **Use a blank or representative template**, not a heavily-filled instance — the fingerprint keys off the form's **boilerplate text**, so a clean template with the full boilerplate gives the strongest, most specific fingerprint.

• **Prefer form-specific text over generic letterhead.** If a form's only distinctive text is a common company header, its fingerprint can over-match unrelated documents that share that header. Choose the copy with the most form-unique boilerplate; if needed, trim generic chrome.

• **Confirm the template is text-based** (selectable text, not a flattened image) and inside Microsoft's supported **type/size** limits before fingerprinting — an image-only or oversized file yields no or a poor fingerprint. **Verify current limits for your tenant.**

Simulate before you enforce — always

• **Never** switch the DLP policy carrying a fingerprint straight to enforce. Run it in **simulation** first, long enough to capture representative form traffic (Req 2.6), and read the results before enabling.

• Treat **simulation mode as your permanent safety switch**: the enforced policy can be returned to simulation in one setting, instantly removing enforcement with no data change — this is your rollback.

• Prove **both** the positive case (the filled form is detected) **and** the negative case (unrelated content is not) in simulation before enforcing.

Keep detection specific (protect adoption)

• A fingerprint is high-precision by design, but **watch for shared boilerplate** across forms — test that each form is detected on its **own** distinctive text, not just a common header (Testing B3).

• Where one SIT over-matches, **split** it so each form is detected independently; where several truly related forms belong together, **group** their fingerprints in one SIT for simpler policy wiring.

• Every false positive that reaches a user erodes trust in **every** policy tip. Budget for a short tuning/re-sourcing cycle, not a one-time build.

Refresh fingerprints as forms change

• A fingerprint reflects the template **as fingerprinted**. When a form is revised, detection of the new version degrades — **add the revised template's fingerprint to the SIT** (a SIT can hold multiple fingerprints) and retire the old where appropriate.

• Assign a **form owner and a refresh trigger** (Req 2.5) so template changes reliably reach the person who re-fingerprints. Stale fingerprints are a silent coverage gap.

• **Version and document** each fingerprint (form name, template version, owner, date) so refreshes are traceable.

Right-size the user experience

• Show **clear, actionable policy tips** — tell the user *why* and *what to do* in business language, not a compliance error code. Reuse one standard notice (`Fingerprint Notice – Standard – AllPolicies`).

• Prefer **allow-with-justification / override** over hard block for forms where legitimate business sharing exists, then **review the overrides** — routine overriding means the policy is mis-scoped.

• Reserve **hard block** for the forms where compliance/legal require it; escalate to block one form at a time, re-validating each.

Classification hygiene

• **Fingerprinting is one technique** — use it where the *form structure* is the signal. Where a value pattern is the signal use built-in/custom SITs or EDM; where a learned category is the signal use a trainable classifier. Fewer, well-targeted detectors beat a sprawl of overlapping ones.

• The same fingerprint SIT can drive **DLP, auto-labeling, and retention** — get the fingerprint right once and reuse it across the Purview estate.

Operations & governance

• Give fingerprint **DLP alerts an owner and an SLA** (Req 2.7). Match volume without triage is not a control. Forward to SIEM/Sentinel where central SecOps needs visibility.

• Review **Activity Explorer** and the false-positive rate on a **recurring cadence**; re-source or re-scope as forms and behavior drift.

• Enforce **least-privilege roles** and, where possible, **separate SIT/policy authors from alert triagers**.

• Keep a **change log** of every fingerprint created/refreshed and every simulate→enforce transition, with the compliance sign-off attached.

Stay inside the boundary

• Fingerprinting detects **text-pattern** matches — it will not catch an **image-only scan**, a **password-protected** file, or a form edited beyond recognition. It is one control among several, not a guarantee; combine with sensitivity labels, encryption, and other SITs where the risk warrants.

• **Verify licensing/tenant** before relying on any DLP location or action — plans and portal paths evolve; don't assume a feature is present.

• **Which forms are in scope, which obligation applies to each, and the enforced action per form are compliance/records/legal determinations** — you provide the technical control and the evidence; they own the decision.

8. Deliverables Summary — Document Fingerprinting (Microsoft Purview)

What you receive in this solution, and what "done" looks like.

Documentation

Document	What it gives you
Solution Overview	How document fingerprinting works — turning a standard form into a fingerprint SIT, wiring it to DLP, what it can and cannot detect, and the simulate→enforce journey; naming convention.
Requirements Guide	Solution-specific prerequisites (DLP/fingerprint licensing, DLP roles, text-based source templates, form inventory, a form-version refresh process, pilot ring, alert triage owner) each with verify/implement steps, on top of the six category prerequisites.

Document	What it gives you
Implementation Guide	The exact ordered phases: connect/baseline → assemble source forms → create the fingerprint(s) + SIT → build the DLP policy **in simulation** → prove detection → enforce to a pilot ring → expand; each phase named, gated, and with a rollback.
Testing & Validation	Pass/fail gates for the three headline outcomes — forms detected (A), specific / low false-positive (B), incidents handled (C) — plus simulation-first, rollback, and refresh checks, all with fake test data.
Troubleshooting Guide	The common failure modes (fingerprint won't create, form not detected, over-matching, no alerts, refresh drift) and how to resolve them.
Best Practices	Sourcing the right template, simulate-first discipline, keeping detection specific, fingerprint refresh, user-experience design, classification hygiene, and operations/governance.
Executive Summary	One-page business framing for approvers.

PowerShell tools (5) — Security & Compliance PowerShell

All tools are **read-only or `-WhatIf` by default**; writes require an explicit `-Commit`. No credentials are stored in code. They connect via `Connect-IPPSSession`. No cmdlets are invented — all are standard Security & Compliance PowerShell.

#	Script	Role	Default safety
1	**CLSFINGER-Discovery.ps1**	Fingerprint posture assessment: existing document-fingerprint SITs, the DLP policies that reference them and their **mode**, and readiness signals.	Read-only
2	**CLSFINGER-CreateFingerprint.ps1**	Fingerprint/SIT builder: creates a fingerprint from a text-based template (`New-DlpFingerprint`) and creates/updates the fingerprint **SIT** (`New-/Set-DlpSensitiveInformationType`). Handles PS 5.1 and 7 byte reads. Never touches a DLP policy's enforce state.	`-WhatIf` (writes need `-Commit`)
3	**CLSFINGER-Validation.ps1**	Wiring validator: confirms the fingerprint SIT exists, that the DLP policy references it, and that the policy is in **simulation** (or reports its mode).	Read-only

#	Script	Role	Default safety
4	**CLSFINGER-Report.ps1**	Detection / false-positive report: exports DLP detections for the fingerprint policy for TP-vs-FP classification and rate calculation.	Read-only
5	**CLSFINGER-Set-Scope.ps1**	Scope/health helper: reports and (with `-Commit`) adjusts a policy's included/excluded users/groups (e.g. the pilot ring), and reports fingerprint-SIT health.	`-WhatIf` (writes need `-Commit`)

Definition of done

• Every in-scope standard form has a text-based template, fingerprinted into a named **fingerprint SIT** (related forms grouped where sensible).

• A DLP policy references the fingerprint SIT and has been **proven in simulation** — the filled forms detected, unrelated content not.

• False-positive rate measured and at/under the agreed threshold; over-matching resolved by re-sourcing or splitting SITs (Gate B).

• Enforcement live for the pilot ring with policy tips + override, then expanded per compliance's action determination (Gate A).

• DLP alerts raised, triaged to disposition, and evidence trailed in Activity Explorer (Gate C).

• A **fingerprint refresh process** in place for form-version changes; simulation-rollback and safe-by-default scripts verified; change log and compliance sign-offs captured.

> ***Boundary:*** *the solution delivers the technical control and the evidence. The* ***forms in scope, the legal/regulatory obligation for each, and the enforced action per form*** *are compliance/privacy/legal determinations, captured alongside the evidence.*

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

CLSFINGER-CreateFingerprint.ps1

Purpose: Creates a document fingerprint from a text-based form template (New-DlpFingerprint) and creates a new fingerprint sensitive information type, or ADDS the fingerprint to an existing one (New-/Set-DlpSensitiveInformationType). SAFE BY DEFAULT: with no -Commit it only reports what it WOULD do (-WhatIf). It never creates, modifies, or enforces a DLP policy. Use for both first-time fingerprinting and form-version refresh (add the revised template's fingerprint to the same SIT). HOW TO RUN PowerShell 5.1+ or 7+ with Security & Compliance PowerShell (Connect-IPPSSession): Connect-IPPSSession -UserPrincipalName admin@contoso.com Preview (default, no change): .\CLSFINGER-CreateFingerprint.ps1 -TemplatePath .\Onboarding-Template.docx -SitName "Standard Forms - Fingerprint SIT - Classification" Actually create/update the SIT: .\CLSFINGER-CreateFingerprint.ps1 -TemplatePath .\Onboarding-Template.docx -SitName "Standard Forms - Fingerprint SIT - Classification" -Commit EXPECTED RESULTS Without -Commit: prints the fingerprint description and whether the SIT would be created or updated. With -Commit: the fingerprint SIT exists / carries the new fingerprint (confirm with CLSFINGER-Validation.ps1). Non-text/image-only/oversized templates yield a poor or failed fingerprint — the script validates the file exists and warns. .NOTES White-label Microsoft Purview tooling. -WhatIf by default; writes require -Commit. No credentials stored. Cmdlets (New-DlpFingerprint, New-/Set-/Get-DlpSensitiveInformationType) are standard Security & Compliance PowerShell; none are invented. Supported file types/size limits are licensing-/tenant-dependent — verify current limits for your tenant. > [CmdletBinding()] param([Parameter(Mandatory = $true)][string]$TemplatePath, [Parameter(Mandatory = $true)][string]$SitName, [string]$FingerprintDescription, [string]$SitDescription = "Detects standard forms/templates via document fingerprinting.", [switch]$Commit)

How to run: PowerShell 5.1+ or 7+ with Security & Compliance PowerShell (Connect-IPPSSession): Connect-IPPSSession -UserPrincipalName admin@contoso.com Preview (default, no change): .\CLSFINGER-CreateFingerprint.ps1 -TemplatePath .\Onboarding-Template.docx -SitName "Standard Forms - Fingerprint SIT - Classification" Actually create/update the SIT: .\CLSFINGER-CreateFingerprint.ps1 -TemplatePath .\Onboarding-Template.docx -SitName "Standard Forms - Fingerprint SIT - Classification" -Commit EXPECTED RESULTS Without -Commit: prints the fingerprint description and whether the SIT would be created or updated. With -Commit: the fingerprint SIT exists / carries the new fingerprint (confirm with CLSFINGER-Validation.ps1). Non-text/image-only/oversized templates yield a poor or failed fingerprint — the script validates the file exists and warns. .NOTES White-label Microsoft Purview tooling. -WhatIf by default; writes require -Commit. No credentials stored. Cmdlets (New-DlpFingerprint, New-/Set-/Get-DlpSensitiveInformationType) are standard Security & Compliance PowerShell; none are invented. Supported file types/size limits are licensing-/tenant-dependent — verify current limits for your tenant. > [CmdletBinding()] param([Parameter(Mandatory = $true)][string]$TemplatePath, [Parameter(Mandatory = $true)][string]$SitName, [string]$FingerprintDescription, [string]$SitDescription = "Detects standard forms/templates via document fingerprinting.", [switch]$Commit)

CLSFINGER-Discovery.ps1

Purpose: Read-only posture assessment for Document Fingerprinting. Lists the sensitive information types in the tenant (flagging likely document-fingerprint SITs), the DLP policies and rules and their MODE (simulation vs enforced), and highlights which fingerprint SITs are — and are not — referenced by a DLP policy. Establishes the "what fingerprinting exists today" baseline before you build. Changes nothing. HOW TO RUN PowerShell 5.1+ or 7+ with Security & Compliance PowerShell (Connect-IPPSSession): Connect-IPPSSession -UserPrincipalName admin@contoso.com .\CLSFINGER-Discovery.ps1 .\CLSFINGER-Discovery.ps1 -CsvPath .\CLSFINGER-Posture.csv EXPECTED RESULTS A console summary plus (with -CsvPath) a CSV of SITs, DLP policies/rules, and their modes. A fingerprint SIT not referenced by any DLP policy, or a policy left in a non-simulation mode without a pilot, is itself an early finding. .NOTES White-label Microsoft Purview tooling. Read-only. No credentials stored. Cmdlets are standard Security & Compliance PowerShell; none are invented. Feature/label availability is licensing-dependent — the script warns where something is not readable rather than assuming. > [CmdletBinding()] param([string]$CsvPath)

How to run: PowerShell 5.1+ or 7+ with Security & Compliance PowerShell (Connect-IPPSSession): Connect-IPPSSession -UserPrincipalName admin@contoso.com .\CLSFINGER-Discovery.ps1 .\CLSFINGER-Discovery.ps1 -CsvPath .\CLSFINGER-Posture.csv EXPECTED RESULTS A console summary plus (with -CsvPath) a CSV of SITs, DLP policies/rules, and their modes. A fingerprint SIT not referenced by any DLP policy, or a policy left in a non-simulation mode without a pilot, is itself an early finding. .NOTES White-label Microsoft Purview tooling. Read-only. No credentials stored. Cmdlets are standard Security & Compliance PowerShell; none are invented. Feature/label availability is licensing-dependent — the script warns where something is not readable rather than assuming. > [CmdletBinding()] param([string]$CsvPath)

CLSFINGER-Report.ps1

Purpose: Read-only detection report for the Document Fingerprinting solution. Pulls DLP detection activity so you can classify each match as true positive vs false positive and compute the false-positive rate that gates enforcement (Testing Gate B). Uses Get-DlpDetectionsReport where available; if that reporting cmdlet is not available in your session it falls back to a unified-audit-log query for DLP record types. Changes nothing. HOW TO RUN PowerShell 5.1+ or 7+ with Security & Compliance PowerShell (Connect-IPPSSession): Connect-IPPSSession -UserPrincipalName admin@contoso.com .\CLSFINGER-Report.ps1 -Days 14 .\CLSFINGER-Report.ps1 -Days 14 -PolicyName "DLP - Fingerprinted Forms - Simulation" -CsvPath .\CLSFINGER-Matches.csv EXPECTED RESULTS A console summary and (with -CsvPath) a CSV of detections (date, policy, workload, SIT/action where available) for TP/FP classification. No matches yet can simply mean the policy needs more dwell time or audit logging is still surfacing events — widen -Days and re-run. .NOTES White-label Microsoft Purview tooling. Read-only. No credentials stored. Cmdlets (Get-DlpDetectionsReport, Search-UnifiedAuditLog) are standard Microsoft reporting cmdlets; none are invented. DLP reporting/audit availability is licensing-/tenant-dependent — the script warns and falls back rather than assuming. > [CmdletBinding()] param([int]$Days = 14, [string]$PolicyName, [string]$CsvPath)

How to run: PowerShell 5.1+ or 7+ with Security & Compliance PowerShell (Connect-IPPSSession): Connect-IPPSSession -UserPrincipalName admin@contoso.com .\CLSFINGER-Report.ps1 -Days 14 .\CLSFINGER-Report.ps1 -Days 14 -PolicyName "DLP - Fingerprinted Forms - Simulation" -CsvPath .\CLSFINGER-Matches.csv EXPECTED RESULTS A console summary and (with -CsvPath) a CSV of detections (date, policy, workload, SIT/action where available) for TP/FP classification. No matches yet can simply mean the policy needs more dwell time or audit logging is still surfacing events — widen -Days and re-run. .NOTES White-label Microsoft Purview tooling. Read-only. No credentials stored. Cmdlets

(Get-DlpDetectionsReport, Search-UnifiedAuditLog) are standard Microsoft reporting cmdlets; none are invented. DLP reporting/audit availability is licensing-/tenant-dependent — the script warns and falls back rather than assuming. > [CmdletBinding()] param([int]$Days = 14, [string]$PolicyName, [string]$CsvPath)

CLSFINGER-Set-Scope.ps1

Purpose: Scope and health helper for the fingerprint DLP policy. Read path: reports the target policy's mode and current Exchange included/excluded scope, and the health of the referenced fingerprint SIT. Write path (only with -Commit): sets the policy's INCLUDED distribution group (the pilot ring) and/or EXCLUDED group for the Exchange location, so enforcement can be constrained to a ring before expansion. SAFE BY DEFAULT: without -Commit it only previews (-WhatIf). It does NOT change a policy's enforce/simulation MODE — that transition stays a deliberate portal action (Impl. Phase 5). HOW TO RUN PowerShell 5.1+ or 7+ with Security & Compliance PowerShell (Connect-IPPSSession): Connect-IPPSSession -UserPrincipalName admin@contoso.com Report only: .\CLSFINGER-Set-Scope.ps1 -PolicyName "DLP - Fingerprinted Forms - Simulation" Preview scoping to the pilot ring (no change): .\CLSFINGER-Set-Scope.ps1 -PolicyName "DLP - Fingerprinted Forms - Simulation" -IncludeGroup "Fingerprint Pilot Ring - Stage1" Apply: .\CLSFINGER-Set-Scope.ps1 -PolicyName "DLP - Fingerprinted Forms - Simulation" -IncludeGroup "Fingerprint Pilot Ring - Stage1" -Commit EXPECTED RESULTS Report: current mode + scope printed. Preview: prints the Set-DlpCompliancePolicy call that WOULD run. Commit: the policy's Exchange included/excluded scope is updated (confirm with CLSFINGER-Validation.ps1 / portal). Mode is never changed by this script. .NOTES White-label Microsoft Purview tooling. -WhatIf by default; writes require -Commit. No credentials stored. Cmdlets (Get-/Set-DlpCompliancePolicy, Get-DlpSensitiveInformationType) are standard Security & Compliance PowerShell; none are invented. Scope parameters shown apply to the Exchange location; other locations are scoped by their own site/account parameters — verify against your tenant. > [CmdletBinding()] param([Parameter(Mandatory = $true)][string]$PolicyName, [string]$IncludeGroup, [string]$ExcludeGroup, [string]$SitName, [switch]$Commit)

How to run: PowerShell 5.1+ or 7+ with Security & Compliance PowerShell (Connect-IPPSSession): Connect-IPPSSession -UserPrincipalName admin@contoso.com Report only: .\CLSFINGER-Set-Scope.ps1 -PolicyName "DLP - Fingerprinted Forms - Simulation" Preview scoping to the pilot ring (no change): .\CLSFINGER-Set-Scope.ps1 -PolicyName "DLP - Fingerprinted Forms - Simulation" -IncludeGroup "Fingerprint Pilot Ring - Stage1" Apply: .\CLSFINGER-Set-Scope.ps1 -PolicyName "DLP - Fingerprinted Forms - Simulation" -IncludeGroup "Fingerprint Pilot Ring - Stage1" -Commit EXPECTED RESULTS Report: current mode + scope printed. Preview: prints the Set-DlpCompliancePolicy call that WOULD run. Commit: the policy's Exchange included/excluded scope is updated (confirm with CLSFINGER-Validation.ps1 / portal). Mode is never changed by this script. .NOTES White-label Microsoft Purview tooling. -WhatIf by default; writes require -Commit. No credentials stored. Cmdlets (Get-/Set-DlpCompliancePolicy, Get-DlpSensitiveInformationType) are standard Security & Compliance PowerShell; none are invented. Scope parameters shown apply to the Exchange location; other locations are scoped by their own site/account parameters — verify against your tenant. > [CmdletBinding()] param([Parameter(Mandatory = $true)][string]$PolicyName, [string]$IncludeGroup, [string]$ExcludeGroup, [string]$SitName, [switch]$Commit)

CLSFINGER-Validation.ps1

Purpose: Read-only validation of the fingerprinting wiring. Confirms the named fingerprint sensitive information type EXISTS, finds every DLP rule/policy that REFERENCES it, and reports each policy's MODE — flagging PASS when the policy is in simulation (the house default before enforcement) and

WARNING when a policy is enforced without evidence of a pilot. Does not test document content (content detection is proven by moving a filled form through a covered channel — see Testing). Changes nothing. HOW TO RUN PowerShell 5.1+ or 7+ with Security & Compliance PowerShell (Connect-IPPSSession): Connect-IPPSSession -UserPrincipalName admin@contoso.com .\CLSFINGER-Validation.ps1 -SitName "Standard Forms - Fingerprint SIT - Classification" .\CLSFINGER-Validation.ps1 -SitName "Standard Forms - Fingerprint SIT - Classification" -CsvPath .\CLSFINGER-Validation.csv EXPECTED RESULTS PASS: SIT exists AND is referenced by at least one DLP policy in simulation mode. WARN: SIT exists but is unreferenced (inert), or a referencing policy is already enforced. FAIL: SIT not found. .NOTES White-label Microsoft Purview tooling. Read-only. No credentials stored. Cmdlets are standard Security & Compliance PowerShell; none are invented. > [CmdletBinding()] param([Parameter(Mandatory = $true)][string]$SitName, [string]$CsvPath)

How to run: PowerShell 5.1+ or 7+ with Security & Compliance PowerShell (Connect-IPPSSession): Connect-IPPSSession -UserPrincipalName admin@contoso.com .\CLSFINGER-Validation.ps1 -SitName "Standard Forms - Fingerprint SIT - Classification" .\CLSFINGER-Validation.ps1 -SitName "Standard Forms - Fingerprint SIT - Classification" -CsvPath .\CLSFINGER-Validation.csv EXPECTED RESULTS PASS: SIT exists AND is referenced by at least one DLP policy in simulation mode. WARN: SIT exists but is unreferenced (inert), or a referencing policy is already enforced. FAIL: SIT not found. .NOTES White-label Microsoft Purview tooling. Read-only. No credentials stored. Cmdlets are standard Security & Compliance PowerShell; none are invented. > [CmdletBinding()] param([Parameter(Mandatory = $true)][string]$SitName, [string]$CsvPath)